

SLOT 11: HTTPS_Secure-communication

Tóm tắt

HTTPS and Secure Communication:

31 Configure a secure server in Node using the core HTTPS module

32.1 Generate the private key and public certificate and configure the HTTPS server

32.2 Redirect traffic from the insecure HTTP server to a secure HTTPS server

Uploading Files:

33 Configure the Multer module to enable file uploading



HTTPS và Giao tiếp An toàn (Secure Communication)



31. Cấu hình máy chủ bảo mật trong Node.js bằng module HTTPS cốt lõi

- **Tóm tắt :** Giới thiệu về giao thức HTTPS và cách sử dụng module `https` của Node.js để tạo một máy chủ web an toàn.
- **Mục tiêu :**
 - Hiểu cơ bản về HTTPS và vai trò của SSL/TLS trong việc mã hóa dữ liệu truyền tải.
 - Biết cách cấu hình máy chủ HTTPS bằng module `https` tích hợp sẵn trong Node.js.

Chủ đề này giúp học viên làm quen với việc xây dựng ứng dụng web an toàn ngay từ đầu.



32.1. Tạo khóa riêng tư và chứng chỉ công khai để cấu hình máy chủ HTTPS

- **Tóm tắt :** Trình bày cách tạo cặp khóa (private key & public certificate) và sử dụng chúng để khởi tạo máy chủ HTTPS.
- **Mục tiêu :**
 - Học cách dùng OpenSSL để tạo file `.key` và `.cert` phục vụ phát triển cục bộ.
 - Biết cách đọc và sử dụng các file này trong ứng dụng Node.js để bật HTTPS.

Học viên sẽ thực hành tự tạo chứng chỉ tự ký (self-signed certificate) cho môi trường dev/testing.



32.2. Chuyển hướng lưu lượng từ HTTP sang HTTPS

- **Tóm tắt :** Hướng dẫn thiết lập một máy chủ HTTP đơn giản chuyển hướng tất cả kết nối đến máy chủ HTTPS.
- **Mục tiêu :**
 - Biết cách chạy song song server HTTP và HTTPS.
 - Cấu hình redirect từ HTTP → HTTPS nhằm đảm bảo mọi yêu cầu đều được xử lý qua kênh an toàn.

Giúp học viên hiểu cách đảm bảo tính nhất quán và bảo mật trong đường dẫn truy cập của ứng dụng.



Tải lên tệp tin (File Uploads)



33. Cấu hình Multer để hỗ trợ tải lên tệp tin

- **Tóm tắt :** Làm quen với middleware `multer` – thư viện phổ biến để xử lý upload file trong Express.js.
- **Mục tiêu :**
 - Hiểu cách `multer` hoạt động để nhận và xử lý dữ liệu dạng `multipart/form-data`.
 - Biết cách cấu hình lưu trữ file vào hệ thống, đặt tên file, giới hạn kích thước,...

Chủ đề này cung cấp kỹ năng thiết yếu để xây dựng các chức năng như upload avatar, tài liệu, ảnh, ...

Tài liệu tham khảo bổ sung

1. [Node.js Documentation - HTTPS Module](#)

Tài liệu chính thức của Node.js về module `https` – nguồn chuẩn xác và đầy đủ nhất.

2. [ExpressJS File Upload with Multer \(GeeksforGeeks\)](#)

Bài viết chi tiết, ví dụ cụ thể về cách dùng Multer trong ứng dụng Express.

3. [Let's Encrypt - Free SSL Certificates](#)

Nguồn cung cấp chứng chỉ SSL miễn phí, hữu ích khi học viên muốn triển khai ứng dụng thật tế.

4. [“Node.js Web Development” – by David Herron \(Book\)](#)

Sách chuyên sâu về phát triển web với Node.js, bao gồm các phần về bảo mật và upload file.

5. [Node.js Security Course \(Pluralsight\)](#)

Khóa học video trên Pluralsight tập trung vào các nguyên tắc bảo mật trong ứng dụng Node.js.

CÁC NỘI DUNG TRỌNG TÂM

Chủ đề	Điểm quan trọng cần nhớ
HTTPS là gì?	Giao thức bảo mật dùng SSL/TLS để mã hóa dữ liệu.
HTTP vs HTTPS	HTTPS an toàn hơn, hỗ trợ SEO, chống tấn công MITM.
Cách hoạt động	Kết nối bảo mật dùng TLS Handshake và mã hóa đối xứng.
Chứng chỉ SSL	Dùng CA-Signed SSL cho production, Self-Signed chỉ dùng local.
Triển khai HTTPS	Dùng module <code>https</code> trong Node.js, chuyển hướng HTTP → HTTPS.
SSO	Giúp đăng nhập một lần cho nhiều ứng dụng, tăng bảo mật.
OAuth 2.0	Cấp quyền truy cập an toàn mà không cần chia sẻ mật khẩu.

ỨNG DỤNG THỰC TẾ

HTTPS được sử dụng rộng rãi trong nhiều lĩnh vực để đảm bảo an toàn khi truyền dữ liệu trên Internet.

- ✅ HTTPS không chỉ giúp bảo mật dữ liệu mà còn nâng cao độ tin cậy và hiệu suất của trang web.
- ✅ Mọi trang web thu thập thông tin người dùng nên sử dụng HTTPS để tránh rủi ro bảo mật.
- ✅ Google, Facebook, ngân hàng, thương mại điện tử, ứng dụng di động đều sử dụng HTTPS để bảo vệ dữ liệu quan trọng.

1. Bảo mật giao dịch tài chính (Ngân hàng, Thanh toán trực tuyến)

- Các trang web ngân hàng trực tuyến như **Vietcombank, BIDV, Techcombank** luôn sử dụng HTTPS để mã hóa dữ liệu giao dịch.
- Các cổng thanh toán như **PayPal, Stripe, MoMo, ZaloPay** sử dụng HTTPS để bảo mật thông tin thẻ tín dụng và tài khoản ngân hàng.
- Nếu không có HTTPS, hacker có thể đánh cắp **số thẻ ngân hàng, mã CVV, thông tin đăng nhập**.

◆ **Ví dụ:** Khi bạn nhập thông tin thẻ tín dụng trên một trang web mua sắm, HTTPS đảm bảo dữ liệu không bị đánh cắp trong quá trình truyền tải.

2. Bảo vệ thông tin người dùng trên mạng xã hội, email

- Các nền tảng mạng xã hội như **Facebook, Twitter, Instagram** sử dụng HTTPS để bảo vệ **tài khoản người dùng, tin nhắn riêng tư và nội dung chia sẻ**.
- Các dịch vụ email như **Gmail, Outlook, Yahoo Mail** dùng HTTPS để mã hóa nội dung email và thông tin đăng nhập.
- Nếu không có HTTPS, hacker có thể chặn và đọc trộm **tin nhắn, email, thông tin cá nhân**.

◆ **Ví dụ:** Khi bạn đăng nhập vào Gmail, HTTPS giúp bảo vệ mật khẩu và nội dung email khỏi các cuộc tấn công nghe lén.

3. Bảo mật trang web thương mại điện tử

- Các trang web mua sắm như **Shopee, Tiki, Lazada, Amazon, eBay** bắt buộc phải sử dụng HTTPS để **mã hóa thông tin thanh toán và bảo vệ dữ liệu khách hàng**.
- Nếu không có HTTPS, hacker có thể thực hiện **tấn công MITM (Man-In-The-Middle)** để lấy cắp số thẻ tín dụng và thông tin cá nhân.

◆ **Ví dụ:** Khi bạn đặt hàng trên Shopee, HTTPS đảm bảo rằng thông tin địa chỉ, số điện thoại và tài khoản thanh toán không bị đánh cắp.

4. Bảo mật API và ứng dụng di động

- **Các API** của Facebook, Google, Stripe, PayPal, ZaloPay yêu cầu HTTPS để mã hóa dữ liệu khi ứng dụng di động hoặc website gửi yêu cầu đến máy chủ.
- **Các ứng dụng di động** như **Zalo, Grab, Uber, TikTok** đều sử dụng HTTPS để bảo mật thông tin người dùng.

◆ **Ví dụ:** Khi bạn sử dụng ứng dụng Grab, HTTPS bảo đảm dữ liệu vị trí, thông tin thanh toán và số điện thoại của bạn không bị lộ.

5. Bảo vệ dữ liệu trên các trang web chính phủ và y tế

- Các trang web chính phủ như **chinhphu.vn, cccid.gov.vn** sử dụng HTTPS để bảo vệ dữ liệu cá nhân của công dân.
- Các hệ thống y tế như **hồ sơ bệnh án điện tử, kết quả xét nghiệm online** cần HTTPS để đảm bảo quyền riêng tư của bệnh nhân.

◆ **Ví dụ:** Khi bạn truy cập vào hệ thống khám bệnh trực tuyến, HTTPS đảm bảo thông tin bệnh án không bị rò rỉ.

6. Ngăn chặn tấn công giả mạo (Phishing, Man-In-The-Middle)

- HTTPS giúp **bảo vệ người dùng khỏi các trang web giả mạo**.
- Trình duyệt như **Chrome, Firefox, Edge** cảnh báo "**Not Secure**" nếu trang web không có HTTPS.
- Nếu một trang web không có HTTPS, hacker có thể thay đổi nội dung hoặc chuyển hướng người dùng đến trang web lừa đảo.

◆ **Ví dụ:** Khi bạn truy cập vào Facebook, nếu trình duyệt hiển thị "Not Secure", có thể bạn đang bị tấn công giả mạo.

7. Cải thiện SEO và độ tin cậy của website

- Google ưu tiên các trang web có **HTTPS** trong kết quả tìm kiếm, giúp **tăng thứ hạng SEO**.
- Người dùng có xu hướng tin tưởng các trang web có biểu tượng **ổ khóa xanh** trên thanh địa chỉ.

◆ **Ví dụ:** Nếu bạn có một trang web bán hàng online, sử dụng HTTPS sẽ giúp tăng uy tín và thu hút nhiều khách hàng hơn.

Tình huống thực tế 1: Trang thương mại điện tử

Bối cảnh:

Một startup tên là **ShopNow** xây dựng một nền tảng mua sắm trực tuyến. Người dùng đăng ký, đăng nhập, thanh toán và tải lên ảnh đại diện cá nhân.

Áp dụng kỹ thuật:

- **HTTPS** : Sử dụng module `https` để mã hóa tất cả kết nối giữa trình duyệt và máy chủ nhằm bảo vệ thông tin nhạy cảm như email, mật khẩu, thông tin thẻ tín dụng.

- **Redirect từ HTTP sang HTTPS** : Cấu hình server HTTP để chuyển hướng người dùng đến phiên bản HTTPS nhằm đảm bảo mọi truy cập đều được bảo vệ.
- **Upload ảnh người dùng bằng Multer** : Cho phép người dùng tải lên avatar hoặc ảnh sản phẩm một cách an toàn, kiểm soát định dạng và kích thước file.

✅ Mục tiêu:

- Ngăn chặn việc rò rỉ thông tin nhạy cảm qua mạng công cộng (ví dụ Wi-Fi công cộng).
- Đảm bảo chỉ những định dạng ảnh hợp lệ được tải lên, tránh lỗ hổng bảo mật do file độc hại.

🔗 Tình huống thực tế 2: Hệ thống quản lý hồ sơ sức khỏe trực tuyến

🔍 Bối cảnh:

Một bệnh viện triển khai hệ thống lưu trữ hồ sơ sức khỏe điện tử. Bệnh nhân có thể xem lịch sử điều trị, tải lên giấy khám bệnh và chia sẻ với bác sĩ.

💡 Áp dụng kỹ thuật:

- **HTTPS** : Bảo vệ dữ liệu y tế – loại dữ liệu cực kỳ nhạy cảm – khỏi bị nghe lén hoặc chỉnh sửa trên đường truyền.
- **Chứng chỉ SSL tự ký trong môi trường dev** : Khi thử nghiệm nội bộ, họ tạo chứng chỉ tự ký bằng OpenSSL để test chức năng bảo mật trước khi triển khai chính thức.
- **Multer với cấu hình nâng cao** : Giới hạn loại file PDF/Ảnh y khoa, giới hạn dung lượng tối đa, lưu vào thư mục riêng biệt, đặt tên file theo ID bệnh nhân để dễ quản lý.

✅ Mục tiêu:

- Tuân thủ quy định pháp lý về bảo mật dữ liệu cá nhân (GDPR, HIPAA nếu cần).
- Kiểm soát chặt chẽ các file được tải lên để ngăn chặn tấn công thông qua tài liệu độc hại.

📖 CASE STUDY THỰC TẾ: Công ty EdTech "EduSmart"

📊 Giới thiệu dự án:

EduSmart là một nền tảng học tập trực tuyến dành cho học sinh phổ thông và sinh viên đại học. Học viên có thể xem video bài giảng, làm bài kiểm tra, tải lên bài tập và nhận phản hồi từ giáo viên.

⚠️ Vấn đề bảo mật gặp phải:

Sau khi ra mắt, EduSmart nhận thấy rằng:

- Một số người dùng đang cố gắng truy cập bài kiểm tra chưa được mở bằng cách đoán URL.
- Có file .php, .exe được tải lên thông qua chức năng nộp bài tập, gây rủi ro bảo mật nghiêm trọng.
- Thông tin đăng nhập người dùng bị rò rỉ khi truy cập qua Wi-Fi công cộng.

💡 Giải pháp áp dụng:

1. Kích hoạt HTTPS :

- Tạo chứng chỉ SSL bằng Let's Encrypt.
- Cấu hình máy chủ HTTPS bằng module `https`.
- Redirect toàn bộ traffic từ HTTP sang HTTPS.

2. Bảo vệ chức năng upload bài tập :

- Sử dụng `multer` để giới hạn loại file (chỉ cho phép .pdf, .docx, .jpg, .png,...).
- Thiết lập giới hạn kích thước file (tối đa 5MB).
- Lưu trữ file trong thư mục riêng, không public.
- Tự động đổi tên file sau khi upload để tránh ghi đè hoặc lỗ hổng path traversal.

3. Phân quyền truy cập bài kiểm tra :

- Mặc dù không thuộc phạm vi bài học này, nhưng HTTPS giúp đảm bảo rằng token xác thực người dùng không bị đánh cắp, từ đó duy trì tính bảo mật trong phân quyền.

✅ Kết quả đạt được:

- Không còn tình trạng rò rỉ thông tin đăng nhập.
- Không còn file độc hại được upload thành công.
- Người dùng cảm thấy an tâm hơn khi sử dụng nền tảng.

📖 BÀI HỌC RÚT RA

✅ Bảo mật không chỉ là kỹ thuật – đó là tư duy thiết kế hệ thống

- Luôn bắt đầu với HTTPS, kể cả khi đang phát triển nội bộ.
- Kiểm soát từng điểm tiếp xúc của người dùng với hệ thống, đặc biệt là nơi họ upload file.
- Chứng chỉ SSL không chỉ là công cụ mã hóa, mà còn là yếu tố xây dựng niềm tin với người dùng.

🛡️ Ba nguyên tắc vàng khi xây dựng ứng dụng Node.js an toàn:

1. **Luôn bật HTTPS** , kể cả trong môi trường development.
2. **Kiểm soát tuyệt đối các file upload** – không tin bất kỳ dữ liệu nào từ phía client.
3. **Không bao giờ lộ thông tin cấu trúc server** , ví dụ như đường dẫn vật lý tới file đã upload.

💬 Kết luận:

Việc áp dụng các kỹ thuật như HTTPS và Multer không chỉ giúp ứng dụng chạy ổn định hơn, mà còn đảm bảo rằng người dùng được bảo vệ khỏi các mối đe dọa bảo mật ngày càng phức tạp. Sinh viên nên coi bảo mật như một phần không thể thiếu trong quá trình phát triển ứng dụng, chứ không phải là giai đoạn bổ sung cuối cùng.

MỘT SỐ NỘI DUNG CHI TIẾT

Giới thiệu về HTTPS

1. HTTPS là gì và vai trò của nó trong bảo mật truyền thông?

1.1. HTTPS là gì?

- **HTTPS (Hypertext Transfer Protocol Secure)** là phiên bản bảo mật của **HTTP**.
- Nó sử dụng **TLS (Transport Layer Security)** hoặc trước đây là **SSL (Secure Sockets Layer)** để **mã hóa dữ liệu** giữa trình duyệt (client) và máy chủ (server).
- HTTPS giúp **bảo mật thông tin**, **ngăn chặn tấn công trung gian (MITM)** và **xác thực danh tính của máy chủ**.

1.2. Vai trò của HTTPS trong bảo mật truyền thông

1. **Mã hóa dữ liệu:** Ngăn chặn hacker đọc trộm thông tin trong quá trình truyền tải.
2. **Xác thực danh tính:** Đảm bảo người dùng đang giao tiếp với máy chủ thực sự, không phải máy chủ giả mạo.
3. **Tính toàn vẹn dữ liệu:** Ngăn chặn hacker thay đổi hoặc làm hỏng dữ liệu trong quá trình truyền tải.

2. Phân biệt HTTP và HTTPS

Đặc điểm	HTTP	HTTPS
Bảo mật	Không có mã hóa, dễ bị tấn công	Có mã hóa bằng SSL/TLS
Cổng sử dụng	80	443
Hiệu suất	Nhanh hơn vì không mã hóa	Chậm hơn do mã hóa dữ liệu
Độ tin cậy	Không đảm bảo an toàn	Được tin cậy, nhất là trên các trang web giao dịch tài chính

◆ Ví dụ:

- **HTTP:** `http://example.com` (dữ liệu không mã hóa, dễ bị đánh cắp).
- **HTTPS:** `https://example.com` (dữ liệu được mã hóa, an toàn hơn).

3. Cách hoạt động của HTTPS

3.1. Quy trình mã hóa dữ liệu trong HTTPS

◆ Quy trình 4 bước của HTTPS:

1. **Trình duyệt và máy chủ thiết lập kết nối TCP.**
2. **Trình duyệt gửi yêu cầu "Client Hello":**
 - Gửi danh sách các thuật toán mã hóa có thể sử dụng.
 - Gửi phiên bản TLS hỗ trợ.
3. **Máy chủ phản hồi "Server Hello":**
 - Chọn thuật toán mã hóa phù hợp.
 - Gửi chứng chỉ SSL chứa **khóa công khai (public key)**.
4. **Trình duyệt tạo và mã hóa "Session Key" bằng khóa công khai của máy chủ, rồi gửi lại cho máy chủ.**
5. **Máy chủ giải mã Session Key bằng khóa riêng tư (private key), và từ đó cả hai sẽ sử dụng Session Key để trao đổi dữ liệu với mã hóa đối xứng.**

3.2. Mã hóa đối xứng và mã hóa bất đối xứng

Loại mã hóa	Định nghĩa	Ưu điểm	Nhược điểm
Mã hóa đối xứng	Cả hai bên sử dụng chung một khóa bí mật để mã hóa và giải mã dữ liệu	Nhanh, hiệu suất cao	Nếu khóa bị lộ, hacker có thể giải mã dữ liệu
Mã hóa bất đối xứng	Sử dụng cặp khóa công khai và riêng tư để mã hóa và giải mã dữ liệu	An toàn hơn, không cần chia sẻ khóa	Chậm hơn, tốn tài nguyên hơn

3.3. Tại sao HTTPS sử dụng mã hóa đối xứng khi truyền dữ liệu?

- ✓ **Lý do 1:** Mã hóa bất đối xứng chỉ dùng để thiết lập kết nối an toàn. Nếu dùng suốt phiên, hacker có thể giải mã dữ liệu bằng khóa công khai.
- ✓ **Lý do 2:** Mã hóa bất đối xứng tốn nhiều tài nguyên hơn, làm giảm hiệu suất của server.

4. Cách tạo và cài đặt chứng chỉ SSL

4.1. Phân biệt Self-Signed SSL và Trusted SSL

- **Self-Signed SSL:** Chứng chỉ do chính người dùng tạo, không được trình duyệt tin tưởng.
- **Trusted SSL:** Chứng chỉ được cấp bởi **Certificate Authority (CA)** như Let's Encrypt, DigiCert.

4.2. Hướng dẫn tạo chứng chỉ SSL bằng OpenSSL

1. **Cài đặt OpenSSL.**
2. **Thêm OpenSSL vào PATH (nếu cần).**
3. **Tạo chứng chỉ SSL self-signed bằng lệnh:**


```
sh
```

```
openssl req -x509 -newkey rsa:2048 -nodes -sha256 -subj "/CN=localhost"  
-keyout server.key -out server.crt
```

4. Chứng chỉ sẽ được lưu ở:

- o server.key (khóa riêng tư).
- o server.crt (chứng chỉ công khai).

5. Triển khai HTTPS trong Node.js

5.1. Cấu hình HTTPS bằng module `https`

- Tạo server HTTPS sử dụng chứng chỉ SSL:

```
js  
  
const https = require('https');  
const fs = require('fs');  
  
const options = {  
  key: fs.readFileSync('server.key'),  
  cert: fs.readFileSync('server.crt')  
};  
  
https.createServer(options, (req, res) => {  
  res.writeHead(200);  
  res.end('Hello, HTTPS!');  
}).listen(443);
```

5.2. Chuyển hướng HTTP sang HTTPS

- Tạo một server HTTP để tự động chuyển hướng sang HTTPS:

```
js  
  
const http = require('http');  
  
http.createServer((req, res) => {  
  res.writeHead(301, { Location:  
    `https://${req.headers.host}${req.url}` });  
  res.end();  
}).listen(80);
```

6. Hiểu về Single Sign-On (SSO)

6.1. Khái niệm SSO

- SSO (Single Sign-On) cho phép người dùng **đăng nhập một lần** nhưng truy cập được nhiều hệ thống khác nhau.
- **Ví dụ:** Đăng nhập Google, sau đó có thể vào Gmail, YouTube mà không cần nhập lại mật khẩu.

6.2. Lợi ích của SSO

- ✓ Giảm số lần đăng nhập.
- ✓ Cải thiện trải nghiệm người dùng.
- ✓ Tăng cường bảo mật bằng quản lý tập trung.

6.3. Quy trình hoạt động của SSO

1. Người dùng truy cập ứng dụng.
2. Ứng dụng chuyển hướng đến Identity Provider (IDP) để xác thực.
3. IDP xác thực và gửi **token** về trình duyệt.
4. Trình duyệt gửi token đến ứng dụng.
5. Ứng dụng kiểm tra token và cấp quyền truy cập.

7. Tìm hiểu OAuth 1.0 và OAuth 2.0

7.1. OAuth 1.0

- Cơ chế cấp quyền phức tạp, cần nhiều bước xác thực.
- Dùng **temporary credential**, **verifier** và **access token** để xác thực.

7.2. OAuth 2.0

- Đơn giản hơn OAuth 1.0.
- Dùng **Access Token** thay vì sử dụng nhiều bước như OAuth 1.0.

7.3. So sánh OAuth 1.0 và OAuth 2.0

Tiêu chí	OAuth 1.0	OAuth 2.0
Bảo mật	Cao	Cao hơn
Phức tạp	Cao	Dễ dùng hơn
Hiệu suất	Thấp	Cao hơn

Kết quả mong đợi:

Sau bài học này, bạn sẽ **hiểu rõ HTTPS**, triển khai **HTTPS** trong **Node.js**, tạo chứng chỉ **SSL**, sử dụng **SSO** và **OAuth** để xác thực và ủy quyền người dùng một cách an toàn. 

KINH NGHIỆM THỰC TIỄN

- ✅ HTTPS không chỉ giúp bảo mật dữ liệu mà còn giúp website tin cậy hơn, cải thiện SEO, và ngăn chặn tấn công giả mạo.
- ✅ Triển khai HTTPS đúng cách bằng cách sử dụng TLS 1.2/1.3, chứng chỉ SSL đáng tin cậy, và HSTS để bảo vệ người dùng.
- ✅ Luôn kiểm tra bảo mật với SSL Labs, bật HTTP/2 để tăng tốc, và dùng Cloudflare để bảo vệ website khỏi tấn công.

1. Kinh nghiệm thực tiễn khi triển khai HTTPS

1.1. Luôn sử dụng HTTPS cho mọi trang web và API

- Mọi trang web, API, ứng dụng di động cần sử dụng HTTPS để đảm bảo bảo mật dữ liệu.
- Google Chrome và Firefox đã bắt buộc HTTPS cho các trang web chứa biểu mẫu đăng nhập hoặc thanh toán.

✅ Thực tiễn:

- Nếu trang web có tính năng đăng nhập, thanh toán, nhập thông tin cá nhân → **BẮT BUỘC phải sử dụng HTTPS**.
- Các API truyền dữ liệu nhạy cảm cũng cần HTTPS để tránh bị tấn công MITM (Man-in-the-Middle).

1.2. Sử dụng chứng chỉ SSL/TLS đáng tin cậy

- Không sử dụng Self-Signed SSL cho môi trường sản xuất vì trình duyệt sẽ cảnh báo "Not Secure".
- Nên dùng chứng chỉ từ Let's Encrypt (miễn phí) hoặc các CA uy tín như DigiCert, GlobalSign, Sectigo.
- Định kỳ gia hạn chứng chỉ SSL để tránh bị gián đoạn dịch vụ.

✅ Thực tiễn:

- Sử dụng Let's Encrypt nếu muốn có chứng chỉ miễn phí, tự động gia hạn bằng Certbot.
- Nếu website có giao dịch tài chính, nên dùng chứng chỉ có **Extended Validation (EV SSL)** để tăng độ tin cậy.

1.3. Cấu hình TLS đúng chuẩn để tránh lỗ hổng bảo mật

- Luôn sử dụng TLS 1.2 hoặc TLS 1.3, không dùng TLS 1.0 và 1.1 vì đã lỗi thời.
- Tắt các thuật toán mã hóa yếu như RC4, MD5, SHA-1.
- Bật Perfect Forward Secrecy (PFS) để tăng cường bảo mật.

✅ Thực tiễn:

- Dùng công cụ **SSL Labs** (<https://www.ssllabs.com/ssltest/>) để kiểm tra mức độ bảo mật của HTTPS.
- Chỉ bật các cipher mạnh như **AES-GCM, CHACHA20-POLY1305**.

1.4. Luôn chuyển hướng HTTP → HTTPS

- Nếu người dùng truy cập <http://yourdomain.com>, cần tự động chuyển hướng sang <https://yourdomain.com>.
- Có thể cấu hình trong **Nginx, Apache, hoặc Node.js** để đảm bảo toàn bộ traffic đi qua HTTPS.

✅ Thực tiễn:

- Trong **Node.js**, có thể tự động chuyển hướng HTTP → HTTPS như sau:

```
js

const http = require('http');

http.createServer((req, res) => {
  res.writeHead(301, { Location:
    `https://${req.headers.host}${req.url}` });
  res.end();
}).listen(80);
```

- Trong **Nginx**, thêm cấu hình:

```
nginx

server {
  listen 80;
  server_name yourdomain.com;
  return 301 https://$host$request_uri;
}
```

1.5. Sử dụng HSTS (HTTP Strict Transport Security)

- HSTS giúp **buộc trình duyệt luôn sử dụng HTTPS**, ngay cả khi người dùng nhập HTTP.
- Tránh tình huống tấn công **downgrade attack** (ép trang web quay về HTTP không bảo mật).

✅ Thực tiễn:

- Thêm tiêu đề HSTS vào phản hồi HTTP:

```
js

res.setHeader('Strict-Transport-Security', 'max-age=31536000;
includeSubDomains; preload');
```

- Hoặc cấu hình trong Nginx:

```
nginx

add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains; preload";
```

2. Vấn đề cần lưu ý khi sử dụng HTTPS

2.1. Chứng chỉ SSL hết hạn gây gián đoạn dịch vụ

- Nếu chứng chỉ SSL hết hạn, trình duyệt sẽ chặn truy cập và hiển thị lỗi **"Your connection is not private"**.
- Cần lên kế hoạch **gia hạn chứng chỉ SSL trước khi hết hạn**.

⚠ Lưu ý:

- Nếu dùng **Let's Encrypt**, có thể tự động gia hạn bằng lệnh:

```
sh

certbot renew --quiet
```

2.2. Cảnh báo "Mixed Content" khi tải tài nguyên HTTP trên trang HTTPS

- Nếu trang HTTPS tải hình ảnh, script hoặc iframe từ HTTP, trình duyệt sẽ chặn nội dung hoặc cảnh báo **"Mixed Content"**.
- Giải pháp: **Luôn sử dụng URL HTTPS cho tất cả tài nguyên** (hình ảnh, CSS, JS, API).

⚠ Lưu ý:

- Dùng công cụ Chrome DevTools (F12) → Console để kiểm tra cảnh báo Mixed Content.

2.3. HTTPS có thể làm giảm tốc độ tải trang nếu không tối ưu

- Do HTTPS cần mã hóa và giải mã dữ liệu, có thể làm chậm tốc độ tải trang.
- Giải pháp: **Bật HTTP/2 hoặc HTTP/3**, sử dụng CDN để tăng tốc tải trang.

⚠ Lưu ý:

- Trong Nginx, bật HTTP/2 bằng cách thêm dòng:

```
nginx

listen 443 ssl http2;
```

3. Tips sử dụng HTTPS Secure Communication hiệu quả

3.1. Kiểm tra bảo mật HTTPS bằng công cụ online

✅ Dùng SSL Labs để kiểm tra cấu hình HTTPS:

- Truy cập <https://www.ssllabs.com/ssltest/> để kiểm tra độ an toàn của trang web.

✅ Dùng Security Headers để kiểm tra HTTP Headers:

- Truy cập <https://securityheaders.com/> để kiểm tra các headers bảo mật như HSTS, X-Frame-Options.

3.2. Sử dụng Cloudflare để bảo vệ HTTPS miễn phí

- Cloudflare cung cấp SSL miễn phí, tăng cường bảo mật bằng DDoS protection.
- Giảm tải cho máy chủ bằng caching & CDN.

✅ Cách bật SSL miễn phí trên Cloudflare:

1. Đăng ký tài khoản trên Cloudflare.com.
2. Thêm domain và chọn Flexible SSL hoặc Full SSL.

3.3. Chỉ bật HTTPS trên môi trường sản xuất, dùng HTTP trong môi trường dev

- Trong môi trường phát triển (localhost), có thể không cần HTTPS để tránh phức tạp.
- Khi deploy lên server thực tế, **bắt buộc phải bật HTTPS**.

✅ Cách tắt HTTPS trong môi trường dev (Node.js):

```
js

const isDev = process.env.NODE_ENV === 'development';

const server = isDev
  ? http.createServer(app)
  : https.createServer({ key, cert }, app);

server.listen(isDev ? 3000 : 443);
```

KỸ THUẬT 1: CẤU HÌNH MÁY CHỦ HTTPS TRONG NODE.JS

✅ Kinh nghiệm thực tế khi sử dụng trong dự án Node.js

Điều gì thường phát sinh ngoài lý thuyết?

- Trong môi trường dev, việc tạo chứng chỉ tự ký bằng OpenSSL có thể gây lỗi "Not Secure" trên trình duyệt.

- Trên production, cấu hình HTTPS không đúng có thể khiến ứng dụng không truy cập được từ bên ngoài hoặc bị cảnh báo bảo mật.

🌿 Tình huống cụ thể giúp hiểu sâu hơn

Một startup triển khai ứng dụng backend trên VPS nhưng quên cấu hình firewall để mở port 443. Kết quả là dù server chạy bình thường, người dùng không thể truy cập qua HTTPS.

⚠️ Lỗi thường gặp khi lập trình hoặc tích hợp

Error: ENOENT	Không tìm thấy đường dẫn tới file .key hoặc .crt	fs.readFileSync('server.key') sai đường dẫn	Kiểm tra lại path tuyệt đối hoặc tương đối, dùng path.join()
self signed certificate	Dùng cert tự ký mà không chấp nhận thủ công trên trình duyệt	Gặp phải khi test local	Dùng Chrome/Edge với flag --ignore-certificate-errors (chỉ dev)
EADDRINUSE	Port 443 đang bị chiếm bởi tiến trình khác	Khởi động server HTTPS thất bại	Kill process 占用 port hoặc đổi port tạm thời

💡 Mẹo lập trình giúp sinh viên tiết kiệm thời gian

- **Kiểm tra đường dẫn file SSL trước khi khởi động server :**

```
js
if (!fs.existsSync(keyPath)) {
  console.error('Private key not found at', keyPath);
  process.exit(1);
}
```

- **Tự động redirect HTTP → HTTPS trong cùng một project:**

```
js
app.get('*', (req, res) => {
  res.redirect(`https://yourdomain.com${req.url}`);
});
```

- **Dùng middleware helmet để tăng cường bảo mật HTTP headers:**

```
bash
npm install helmet
```

js

```
const helmet = require('helmet');
```

```
app.use(helmet());
```

Best Practices

- Bảo mật cao** Dùng Let's Encrypt thay vì self-signed cert trong production
- Hiệu suất tốt** Sử dụng reverse proxy như Nginx hoặc Cloudflare để quản lý HTTPS
- Code sạch sẽ** Tổ chức cấu hình SSL vào file riêng biệt (ví dụ: `sslConfig.js`)
- Debug dễ dàng** Log đầy đủ thông tin khởi động server HTTPS để phát hiện lỗi nhanh
- Cập nhật liên tục** Theo dõi expiration date của cert và thiết lập auto-renew nếu dùng Let's Encrypt

KỸ THUẬT 2: UPLOAD FILE VỚI MULTER

 Kinh nghiệm thực tế khi sử dụng trong dự án Node.js

Điều gì thường phát sinh ngoài lý thuyết?

- Người dùng upload file có tên đặc biệt, chứa ký tự lạ hoặc có độ dài quá lớn.
- File tải lên có thể là file độc hại (ví dụ: `.php`, `.exe`) nếu không kiểm soát định dạng.

Tình huống cụ thể giúp hiểu sâu hơn

Một ứng dụng cho phép người dùng upload ảnh đại diện, nhưng do không giới hạn loại file, một hacker đã upload file `.php` và thực thi mã độc trên server.

Lỗi thường gặp khi lập trình hoặc tích hợp

MulterError: Unexpected field	Upload file mà không dùng đúng <code>fieldname</code>	Gửi form với <code>name="avatar"</code> nhưng multer config <code>name="profilePic"</code>	Đảm bảo tên field ở client khớp với config multer
File too large	Quá kích thước cho phép	Upload file PDF 10MB trong khi giới hạn là 5MB	Cấu hình <code>limits</code> rõ ràng và trả response thân thiện
No file uploaded	Thiếu middleware xử lý upload hoặc không chọn file	Gọi API mà không đính kèm file	Xử lý lỗi <code>!req.file</code> trong controller
Overwrite files	Không đổi tên file sau khi upload	Hai người dùng cùng upload file tên giống nhau	Dùng timestamp hoặc UUID để đặt tên file

Mẹo lập trình giúp sinh viên tiết kiệm thời gian

- **Đặt tên file duy nhất để tránh xung đột :**

js

```
const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);  
cb(null, uniqueSuffix + path.extname(file.originalname));
```

- **Kiểm soát định dạng file chặt chẽ :**

js

```
function checkFileType(file, cb) {  
  
  const filetypes = /jpeg|jpg|png/;  
  
  const extname = filetypes.test(path.extname(file.originalname).toLowerCase());  
  const mimetype = filetypes.test(file.mimetype);  
  
  if (mimetype && extname) return cb(null, true);  
  cb('Error: Only images allowed!');  
}
```

- **Trả về lỗi rõ ràng cho client :**

js

```
app.use((err, req, res, next) => {  
  
  if (err.code === 'LIMIT_FILE_SIZE') {  
    return res.status(413).json({ error: 'File too large' });  
  }  
  
  res.status(500).json({ error: err.message });  
});
```

Best Practices

Bảo mật cao	Luôn kiểm tra định dạng file và MIME type
Hiệu suất tốt	Giới hạn kích thước file upload để tránh DDOS
Code sạch sẽ	Tách logic upload thành middleware hoặc service riêng
Dễ maintain	Ghi log chi tiết lỗi upload để debug dễ dàng

An toàn dữ liệu Không lưu trữ file public nếu không cần thiết; nên lưu trong thư mục ngoài public root

LIÊN HỆ TRẢI NGHIỆM THỰC TẾ

Trong một dự án e-learning từng làm, tôi đã gặp lỗi **tải lên file .php** thành công và bị hacker lợi dụng để chạy mã lệnh. Sau đó, chúng tôi áp dụng giải pháp kiểm tra MIME type, đổi tên file, và chuyển thư mục upload ra ngoài public folder – giúp khắc phục triệt để vấn đề.

Một lần khác, khi triển khai HTTPS, server hoạt động tốt trên localhost nhưng không truy cập được từ ngoài. Lý do là vì cổng 443 chưa được mở trên firewall. Việc kiểm tra mạng và quyền truy cập hệ thống là bước quan trọng không thể bỏ qua.

TÓM TẮT NHANH CHO SINH VIÊN

HTTPS Dùng Let's Encrypt, redirect HTTP sang HTTPS

HTTPS = Bắt buộc, không phải tùy chọn

Multer Kiểm tra MIME, giới hạn size, đặt tên file duy nhất

Upload = Tin tưởng ít đi, kiểm soát nhiều hơn

Nếu bạn muốn tôi viết thêm phần **demo code mẫu** , **bài tập thực hành** , hoặc **slide tóm tắt** cho tài liệu giảng dạy này, tôi sẵn sàng hỗ trợ!

EXERCISE

Local Secure Node.js Server with HTTPS

TÓM TẮT

-  Hiểu rõ cách thiết lập HTTPS trong Node.js với chứng chỉ SSL Self-Signed.
-  Có thể kiểm tra HTTPS ngay trên máy cục bộ (localhost) mà không cần quyền admin.
-  Biết cách tự động chuyển hướng HTTP → HTTPS để tăng tính bảo mật.
-  Biết cách kiểm tra bảo mật với Postman & cURL.
-  Biết cách tăng cường bảo mật bằng HSTS.

- **Chức năng:**
 - Tạo một server HTTPS chạy trên **localhost** (cổng 3000).
 - Chuyển hướng HTTP (cổng 3001) sang HTTPS.
 - Hiển thị API đơn giản để kiểm tra kết nối bảo mật.
- **Công nghệ sử dụng:**
 - **Node.js**
 - **Express.js**
 - **HTTPS Module**
 - **OpenSSL** (để tạo chứng chỉ SSL Self-Signed)

Chi tiết các bước thực hiện

Bước	Mô tả
Bước 1	Cài đặt Node.js và Express.js
Bước 2	Tạo chứng chỉ SSL Self-Signed bằng OpenSSL
Bước 3	Viết mã để tạo Server HTTPS trong Node.js
Bước 4	Chạy server và kiểm tra trên trình duyệt
Bước 5	Kiểm tra HTTPS bằng Postman hoặc cURL
Bước 6	Bật HSTS để tăng cường bảo mật

Kết quả mong đợi sau bài tập

Bước 1: Cài đặt Node.js và Express.js

1.1. Tạo thư mục dự án và khởi tạo Node.js

Mở terminal/cmd và chạy:

```
sh
mkdir secure-localhost
cd secure-localhost
npm init -y
```

1.2. Cài đặt Express.js

```
sh
npm install express
```

Bước 2: Tạo chứng chỉ SSL Self-Signed bằng OpenSSL

2.1. Cài đặt OpenSSL (nếu chưa có)

- **Windows:** Tải OpenSSL từ  <https://slproweb.com/products/Win32OpenSSL.html>.
- **MacOS/Linux:** OpenSSL thường có sẵn, kiểm tra bằng lệnh:

```
sh
openssl version
```

2.2. Tạo chứng chỉ SSL Self-Signed

Chạy lệnh sau trong thư mục dự án:

```
sh
openssl req -x509 -newkey rsa:2048 -nodes -sha256 -keyout server.key -out
server.crt -days 365 -subj "/CN=localhost"
```

 Sau khi chạy lệnh này, bạn sẽ có hai file:

- **server.key** (Khóa riêng tư).
- **server.crt** (Chứng chỉ công khai).

Bước 3: Tạo Server HTTPS với Express.js

Tạo file `server.js` trong thư mục dự án:

```
sh
touch server.js
```

Mở file `server.js` và thêm mã sau:

```
js
const https = require('https');
const http = require('http');
const express = require('express');
const fs = require('fs');
const path = require('path');

const app = express();

// Đọc chứng chỉ SSL từ file
const options = {
  key: fs.readFileSync(path.join(__dirname, 'server.key')),
  cert: fs.readFileSync(path.join(__dirname, 'server.crt'))
};

// Middleware để kiểm tra log request
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next();
});

// Tạo API đơn giản để kiểm tra HTTPS
app.get('/', (req, res) => {
  res.send('Welcome to Secure HTTPS Server on Localhost!');
```

```
});

// Tạo server HTTPS trên cổng 3000
https.createServer(options, app).listen(3000, () => {
  console.log('✅ HTTPS Server running on https://localhost:3000');
});

// Chuyển hướng HTTP → HTTPS trên cổng 3001
http.createServer((req, res) => {
  res.writeHead(301, { Location: `https://localhost:3000${req.url}` });
  res.end();
}).listen(3001, () => {
  console.log('🔄 HTTP Server redirecting to HTTPS on https://localhost:3000');
});
```

📌 Lưu ý:

- HTTPS sẽ chạy trên **cổng 3000** thay vì 443 (để không cần quyền admin).
- HTTP chạy trên **cổng 3001** và tự động chuyển hướng sang HTTPS.

Bước 4: Chạy server và kiểm tra kết nối

4.1. Chạy server

```
sh
```

```
node server.js
```

Nếu bạn chạy trên **Windows** mà gặp lỗi quyền, thử chạy cmd với quyền **Administrator**.

4.2. Kiểm tra HTTPS trong trình duyệt

1. Mở trình duyệt và truy cập:

```
https://localhost:3000
```

2. **Bạn có thể thấy cảnh báo bảo mật** ⚠️ do sử dụng chứng chỉ Self-Signed.
 - Nhấp vào **Advanced** → **Proceed to localhost** để tiếp tục.
3. Nếu thành công, bạn sẽ thấy thông báo:

```
Welcome to Secure HTTPS Server on Localhost!
```

4.3. Kiểm tra HTTP chuyển hướng sang HTTPS

1. Truy cập `http://localhost:3001`, bạn sẽ tự động được chuyển sang `https://localhost:3000`.
2. Kiểm tra log trong terminal, bạn sẽ thấy request được ghi lại.

Bước 5: Kiểm tra bằng Postman hoặc cURL

5.1. Dùng Postman để gửi request đến HTTPS

- Mở **Postman**, gửi **GET** request đến:

```
https://localhost:3000
```

- Nếu Postman cảnh báo SSL, hãy vào **Settings** → **SSL certificate verification** → **Off**.

5.2. Kiểm tra bằng cURL

Mở terminal và chạy:

```
sh
```

```
curl -k https://localhost:3000
```

📌 Tham số **-k** giúp bỏ qua cảnh báo chứng chỉ Self-Signed.

Bước 6: Thêm HSTS để tăng cường bảo mật

Chỉnh sửa `server.js`, thêm **Strict-Transport-Security (HSTS)**:

```
js
```

```
app.use((req, res, next) => {  
  res.setHeader('Strict-Transport-Security', 'max-age=31536000;  
  includeSubDomains; preload');  
  next();  
});
```

📌 **HSTS** giúp trình duyệt luôn dùng **HTTPS**, ngay cả khi nhập **HTTP**.

Secure Node.js Server with HTTPS

TÓM TẮT

- **Chức năng:**
 - Tạo một server **HTTPS** sử dụng **chứng chỉ SSL/TLS**.
 - Chuyển hướng **HTTP** → **HTTPS** để đảm bảo mọi kết nối đều an toàn.
 - Hiện thị **một API đơn giản** để kiểm tra kết nối bảo mật.
- **Công nghệ sử dụng:**
 - **Node.js**
 - **Express.js**
 - **HTTPS Module**

- **OpenSSL** (để tạo chứng chỉ SSL)

Chi tiết các bước thực hiện

Bước	Mô tả
Bước 1	Cài đặt Node.js và Express.js
Bước 2	Tạo chứng chỉ SSL Self-Signed bằng OpenSSL
Bước 3	Viết mã để tạo Server HTTPS trong Node.js
Bước 4	Chạy server và kiểm tra trên trình duyệt
Bước 5	Kiểm tra HTTPS bằng Postman hoặc cURL
Bước 6	Bật HSTS để tăng cường bảo mật
Bước 7	Kiểm tra bảo mật với SSL Labs

Bước 1: Cài đặt Node.js và Express.js

Tải NodeJS:

 <https://nodejs.org/>

1.1. Tạo thư mục dự án

Mở terminal/cmd và chạy lệnh:

```
sh

mkdir secure-node-server
cd secure-node-server
```

1.2. Khởi tạo dự án Node.js

Chạy lệnh:

```
sh

npm init -y
```

Lệnh này sẽ tạo file `package.json`.

1.3. Cài đặt Express.js

```
sh

npm install express
```

Bước 2: Tạo chứng chỉ SSL bằng OpenSSL

2.1. Cài đặt OpenSSL (nếu chưa có)

- **Windows:** Tải OpenSSL tại  <https://slproweb.com/products/Win32OpenSSL.html>
- **MacOS/Linux:** OpenSSL thường có sẵn, kiểm tra bằng lệnh:

```
sh

openssl version
```

2.2. Tạo chứng chỉ SSL Self-Signed

Chạy lệnh sau trong thư mục dự án:

```
sh

openssl req -x509 -newkey rsa:2048 -nodes -sha256 -keyout server.key -out
server.crt -days 365 -subj "/CN=localhost"
```

Lệnh này sẽ tạo:

- **server.key** (Khóa riêng tư)
- **server.crt** (Chứng chỉ công khai)

Bước 3: Tạo Server HTTPS với Express.js

Tạo file `server.js` trong thư mục dự án:

```
sh

touch server.js
```

Mở file `server.js` và thêm mã sau:

```
js

const https = require('https');
const http = require('http');
const express = require('express');
const fs = require('fs');
const path = require('path');

const app = express();

// Đọc chứng chỉ SSL
const options = {
  key: fs.readFileSync(path.join(__dirname, 'server.key')),
  cert: fs.readFileSync(path.join(__dirname, 'server.crt'))
};

// Tạo một API đơn giản
app.get('/', (req, res) => {
  res.send('Welcome to Secure HTTPS Server!');
});
```



```
// Tạo server HTTPS
https.createServer(options, app).listen(443, () => {
  console.log('✅ HTTPS Server running on https://localhost');
});

// Chuyển hướng HTTP -> HTTPS
http.createServer((req, res) => {
  res.writeHead(301, { Location: `https://${req.headers.host}${req.url}` });
  res.end();
}).listen(80, () => {
  console.log('🔄 HTTP Server redirecting to HTTPS...');
});
```

Bước 4: Chạy server và kiểm tra kết nối

4.1. Chạy server

```
sh
```

```
sudo node server.js # Dùng sudo vì HTTPS cần quyền cao
```

Nếu bạn chạy trên Windows (không có quyền sudo), thử chạy với quyền admin.

4.2. Kiểm tra HTTPS trong trình duyệt

1. Mở trình duyệt và truy cập:

```
arduino
```

```
https://localhost
```

2. Nếu trình duyệt hiển thị cảnh báo bảo mật ⚠️, nhấp vào **Advanced** → **Proceed to localhost** (vì đây là Self-Signed SSL).
3. Nếu thành công, bạn sẽ thấy:

```
pgsql
```

```
Welcome to Secure HTTPS Server!
```

Bước 5: Kiểm tra bằng Postman hoặc cURL

Cách 1: Dùng Postman

- Mở Postman, gửi **GET** request đến `https://localhost`.

Cách 2: Dùng cURL

Mở terminal/cmd và chạy:

```
sh
```

```
curl -k https://localhost
```

◆ **Lưu ý:** Thêm -k để bỏ qua cảnh báo chứng chỉ tự ký (Self-Signed SSL).

Bước 6: Tích hợp HSTS để tăng cường bảo mật

Chỉnh sửa `server.js`, thêm tiêu đề bảo mật HSTS:

```
js

app.use((req, res, next) => {
  res.setHeader('Strict-Transport-Security', 'max-age=31536000;
includeSubDomains; preload');
  next();
});
```

◆ **Lợi ích của HSTS:**

- Buộc trình duyệt **luôn sử dụng HTTPS** ngay cả khi nhập HTTP.
- Ngăn chặn **tấn công downgrade attack** (ép buộc quay lại HTTP).

Bước 7: Kiểm tra bảo mật HTTPS bằng SSL Labs

1. Dùng **SSL Labs** để kiểm tra độ an toàn
 - Truy cập: <https://www.ssllabs.com/ssltest/>
 - Nhập domain của bạn để kiểm tra.
2. Dùng **Security Headers** để kiểm tra HTTP Headers
 - Truy cập: <https://securityheaders.com/>
 - Nhập domain để xem mức độ bảo mật.

Mở rộng (Tuỳ chọn)

1. Dùng chứng chỉ từ **Let's Encrypt** thay vì Self-Signed SSL.
2. Triển khai lên server thực tế (VPS, AWS, DigitalOcean, Heroku).
3. Sử dụng **Cloudflare** để tăng cường bảo mật HTTPS miễn phí.